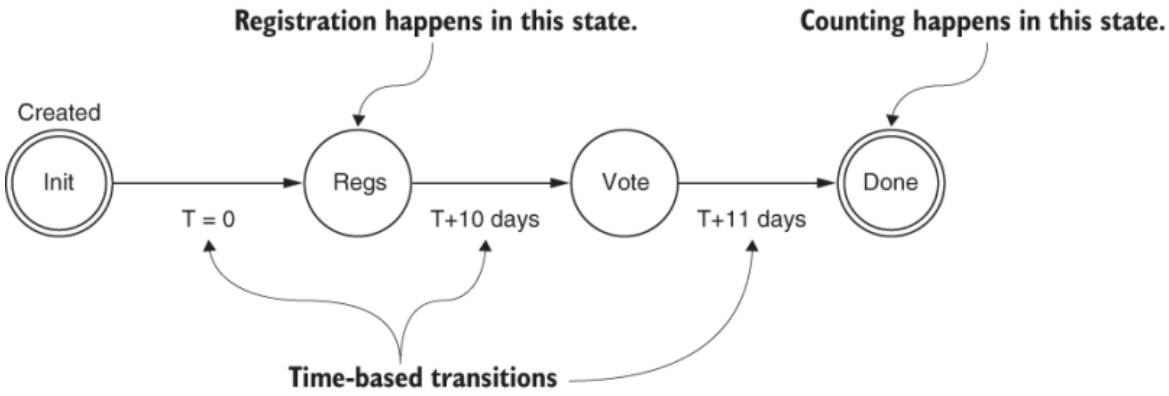


Figure 3.5 FSM design of the ballot state transitions

Now let's translate these design representations into code, as shown in listing 3.2. This listing, BallotV2.sol, has all the contents of BallotV1.sol from listing 3.1, plus a constructor and an additional function: `changeState()`. The enumerated type `Phase` is used for setting the state variable `state`. Your goal is to effect and observe the state changes. Let's assume that the ballot process's chairperson is controlling when the transitions happen by calling the `changeState()` function with a parameter value of `0`, `1`, `2`, or `3` (representing the four phases).

Listing 3.2 Solution with voting states (Ballotv2.sol)

```
// include the code from BallotV1.sol, not shown here

 enum Phase {Init, Regs, Vote, Done}                                    ❶
    // Phase can take only 0,1,2,3 values: Others invalid

 Phase public state = Phase.Init;

 constructor (uint numProposals) public  {                              ❷
        chairperson = msg.sender;                                       ❷
        voters[chairperson].weight = 2; // weight 2 for testing purposes
        for (uint prop = 0; prop < numProposals; prop ++)               ❸
            proposals.push(Proposal(0));                                ❸

    }


    // function for changing Phase: can be done only by chairperson
    function changeState(Phase x) public {                              ❹
```